基于 FPGA 的智能音乐分析终端

摘要

乐器学习中的音高辨认、旋律记录与节拍训练需要直观、及时的反馈。既有研究已形成基于周期性分析的基频估计方法，以及结合倒谱、时序特征的乐器识别方法[8][9]；相关算法在端侧应用时，还需统筹连续采样、频谱计算、模型推理与交互显示。围绕这一需求，本作品设计并实现了基于 FPGA 的智能音乐分析终端，在单块 AC820 开发板上完成声音输入、双路频谱分析、音高与音色识别、简谱显示及节拍输出。

系统充分利用 Zynq-7020 集成可编程逻辑与双核 Cortex-A9 的结构优势：PL 承担连续采集、缓存、加窗及双路 FFT，4096 点幅度谱支撑音高与起音分析，1024 点功率谱支撑三类乐器识别；双路 AXI DMA 将特征送入共享 DDR。PS 通过 PYNQ 调度本地 C 音高算法与 NumPy 卷积网络，并驱动 800×480 触摸界面，实现当前音高、律动频谱、滚动简谱和乐器类别的集中呈现。节拍器由 PL 定时和发声，与分析功能共同构成完整的练习辅助终端。

合并设计在 Zynq-7020 上以 50MHz 主处理时钟完成布局布线，LUT、触发器、BRAM 和 DSP 占用率分别为 26.12%、19.62%、32.14%和 12.27%，建立与保持最差裕量分别为+7.336 ns 和+0.040 ns。通过连续缓存和重叠分帧，系统在保留 4096 点观察窗的同时，将分析更新间隔由 256ms 缩短为 32ms；音色适配采用冻结卷积层、更新 195 个分类头参数的方法，在 55 段分组留出回放中将判对数量由 21 段提高至 42 段。系统以硬件频谱前端和软件分析协同，在片上资源预算内实现音高、音色、简谱与本地交互的集成。

关键词：Zynq-7000、软硬件协同、双路 FFT、音高检测、乐器识别、单旋律简谱

第一部分　作品概述

1.1 功能与特性

系统集音频分析、单旋律记谱与练习交互于一体，主要功能如下。

1. **连续音频采集。**麦克风或 LINE IN 音频经 ES8388 转换为 16 kHz、16 bit 数字采样，I2S 接收器与通道选择逻辑形成单声道数据流；音高支路进行低通滤波，模型支路保留原始 PCM 特征来源。

2. **双路硬件频谱。**4096 点通路以 512 点帧移输出 2049 点幅度谱，兼顾低频观察与连续更新；1024 点通路对每个一秒音频段生成 63 帧、每帧 513 点功率谱，并携带块浮点指数，为模型提供尺度一致的输入。

3. **三类音色识别。**PS 将硬件功率谱映射为 64×63 对数梅尔特征，利用三层卷积网络提取音色特征，结合部署分类头和支持判定，输出钢琴、吉他、小提琴类别及模型概率。

4. **单旋律音符与简谱。**基于完整频谱的 YinFFT 估计基频，起音检测与音符状态机区分音高切换、持续音和同音重奏；音符事件以采样序号记录起止与时长，映射为简谱数字并滚动显示。

5. **触摸显示与硬件节拍。**主页、节拍器、频谱、简谱和乐器识别五个页面集中呈现分析结果。节拍器支持 30～240 BPM 和四种拍号，PL 生成节拍及音频输出，触摸界面完成参数调节与启停。

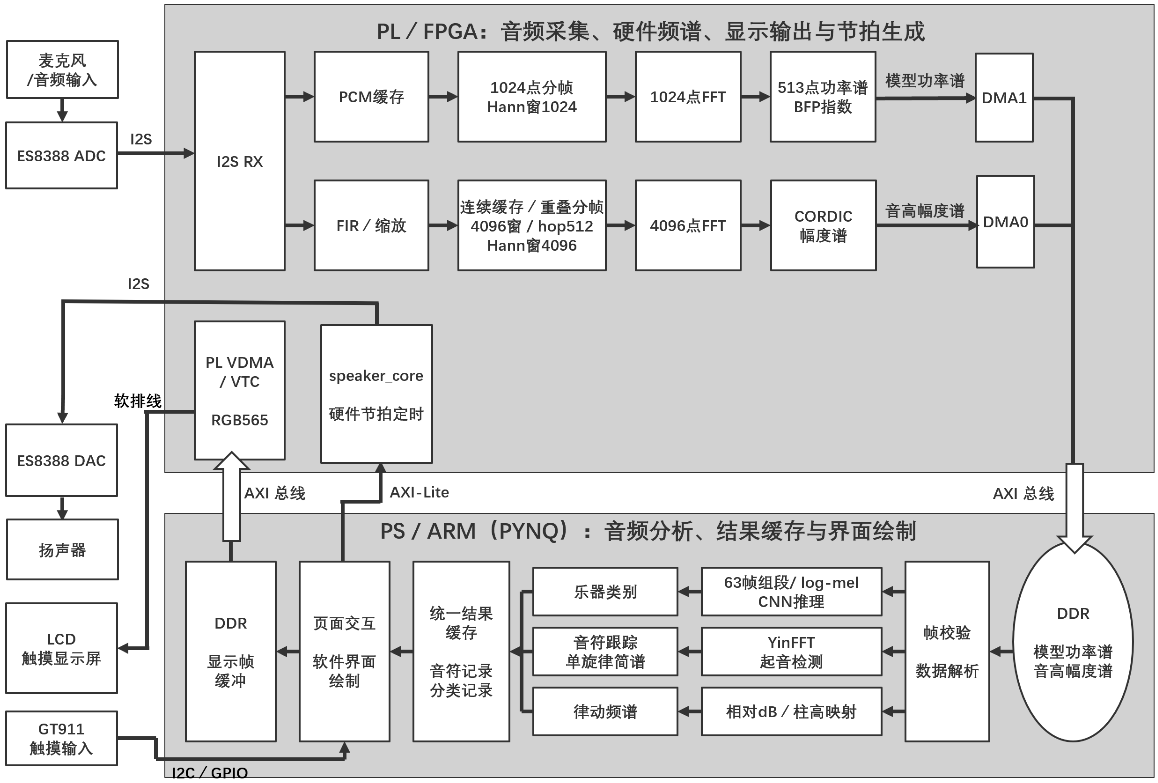


图 1　系统功能总览

1.2 应用领域

主要面向人群：乐器初学者、业余演奏者、音乐教师，以及开展信号处理与软硬件协同实践的学生。

1. **乐器教学与日常练习。**终端将声音变化、当前音高与简谱集中呈现，辅助课堂讲解和课后练习；硬件节拍器为速度保持与节奏训练提供统一参照。

2. **视唱练耳与旋律记录。**当前音符和滚动简谱建立声音与符号的直观联系，三类乐器识别支持音色辨别训练，单旋律事件记录便于回看练习片段。

3. **本地音乐分析。**采集、计算与显示均在开发板本地完成，演示运行无需依赖上位机或网络，适用于琴房、家庭与教学展示等场合。

4. **嵌入式教学与算法实践。**系统贯通音频接口、流式计算、双 DMA、PYNQ、本地算法库和视频输出，可作为软硬件协同设计与信号处理课程的实践平台。

1.3 主要技术特点

1. **单芯片异构协同。**利用 Zynq-7020 的 PL 并行处理能力完成连续采样与频谱前端，利用 Cortex-A9 的控制与软件生态执行基频决策、CNN 和界面调度。计算任务按

数据规律划分，发挥同片集成与共享存储的优势。

2. **面向不同分析目标的双路频谱。**音高支路采用 4096 点长窗与重叠分帧，模型支路采用 1024 点窗与一秒段组装，两路经独立 S2MM 通道接收。独立帧界限和缓冲管理减小业务间的背压耦合，使音高更新与音色分析各按自身节奏推进。

   我们保留两套 FFT，是为了让音高长窗与模型短窗分别按既定格式和节拍工作，简化特征对齐与任务调度。在此基础上，音高、起音和律动显示共用完整幅度谱，分类头与钢琴支持判定共用同一次卷积特征，避免业务功能增加时重复计算。基频决策和模型适配放在 PS，使算法调整复用已有 PL 结构，形成计算规律与软件灵活性之间的分工。

3. **轻量本地部署。**NumPy 实现三层卷积网络前向计算，本地 C 库承担音高与音符运算，PYNQ 负责硬件访问与业务编排。该组合复用既有权重和算法，在 Cortex-A9 上建立紧凑、透明的部署链路。

4. **数据完整性与特征一致性。**系统统一窗函数、帧移、补零、频点数与梅尔矩阵，按帧指数恢复功率尺度；协议中携带段号、样本序号及错误状态，接收端拒绝残缺段，配套清单核对硬件、模型和本地库版本。

1.4 主要性能指标

主要指标按平台、频谱处理、智能分析、交互和实现结果分类汇总。

表格 1　作品主要性能指标

类别	参数项	指标 / 数值	说明
平台	核心器件 / 工具	XC7Z020CLG484-2 / Vivado 2020.2	AC820，PYNQ 本地运行
输入	采样率 / 数据格式	16 kHz / 单声道 16 bit	麦克风或 LINE IN
音高前端	FFT / 窗长 / 帧移	4096 点 / 256 ms / 32ms	2049 点幅度谱，31.25 帧/s（理论值 16000/512）
音高前端	频率网格间隔	3.90625 Hz	采样率除以 FFT 点数
音色前端	FFT / 段长 / 帧移	1024 点 / 1 s / 16 ms	每段 63 帧、513 点功率谱
音色分析	输入 / 输出类别	64×63 log-mel / 3 类	钢琴、吉他、小提琴

类别	参数项	指标 / 数值	说明
数据通路	DMA0 / DMA1 包长	8260 B / 2088 B	两路独立 S2MM
数据通路	平均特征包数据量	约 0.390 MB/s	含包头，按固定帧组织计算
显示交互	LCD / 页面	800×480 RGB565 / 5 页	GT911 触摸
节拍输出	速度 / 拍号	30～240 BPM / 2、3、4、6 拍	界面对应 2/4、3/4、4/4、6/8
实现结果	主处理时钟	50 MHz	合并音频、视频与节拍设计
实现结果	LUT / FF 占用率	26.12% / 19.62%	13897 / 20874 个
实现结果	BRAM / DSP 占用率	32.14% / 12.27%	45 个 36 Kb 等效 / 27 个
实现结果	WNS / WHS	+7.336 ns / +0.040 ns	TNS、THS 均为 0 ns

1.1 主要创新点

1. **架构创新：**双路频谱协同。针对音高分析与音色模型的不同需求，在同一采样源上并行建立长窗幅度谱与短窗功率谱通路，实现精细频率网格、密集时间更新与模型输入约定的兼顾。

2. **接口创新：**带样本时间的双 DMA 数据组织。将帧号、样本序号、数值格式与状态封装进特征包，配合完整段组装和有界队列，使双路特征具备可关联、可检查的时间语义，为音符与音色结果的统一展示提供基础。

3. **部署创新：**固定卷积特征与轻量分类适配。三层卷积提取 64 维音色表示，以小规模分类头适配现场输入；NumPy 前向计算与支持判定复用同次特征，避免重复执行整套卷积计算。

4. **交互创新：**分析与练习交互集成。将当前音、滚动简谱、律动频谱、乐器识别及硬件节拍器整合于本地触摸界面，形成可直接操作的单板终端。

5. **工程方法创新：**可复现的验证链。将协议校验、数值预处理、本地库自检及资源时序检查固化为脚本，并以清单记录源码与产物关系；从模块仿真到整机演示均保留明确的接口与版本依据。

1.2 设计流程

作品采用**自顶向下、分模块验证与逐级集成**的设计流程，流程如图 2 所示。

首先，围绕单旋律分析、三类识别与触摸交互划分 PL/PS 职责，统一采样、窗函数和协议。

随后，分别完成硬件前端与软件算法设计，以采样位置、帧边界和数值格式连接各模块。

验证覆盖有效握手、窗索引、帧长与协议完整性，以已知音源核对频点和连续性。

音高通路采用重叠长窗、YinFFT 和音符状态机；音色通路对齐 log-mel 并完成分类头适配。

统一入口管理双 DMA、算法与界面，经过综合实现、时序检查和整机演示后冻结源码与产物。

Vivado 2020.2 与 AC820 既有工程及 IP 保持一致，提交包提供该工具版本的可复现构建脚本。

大模型协作按**“约束—候选—审阅—验证—纠错”**推进，将基频判定、特征对齐和完整帧接收的经验沉淀为可复用检查规则。

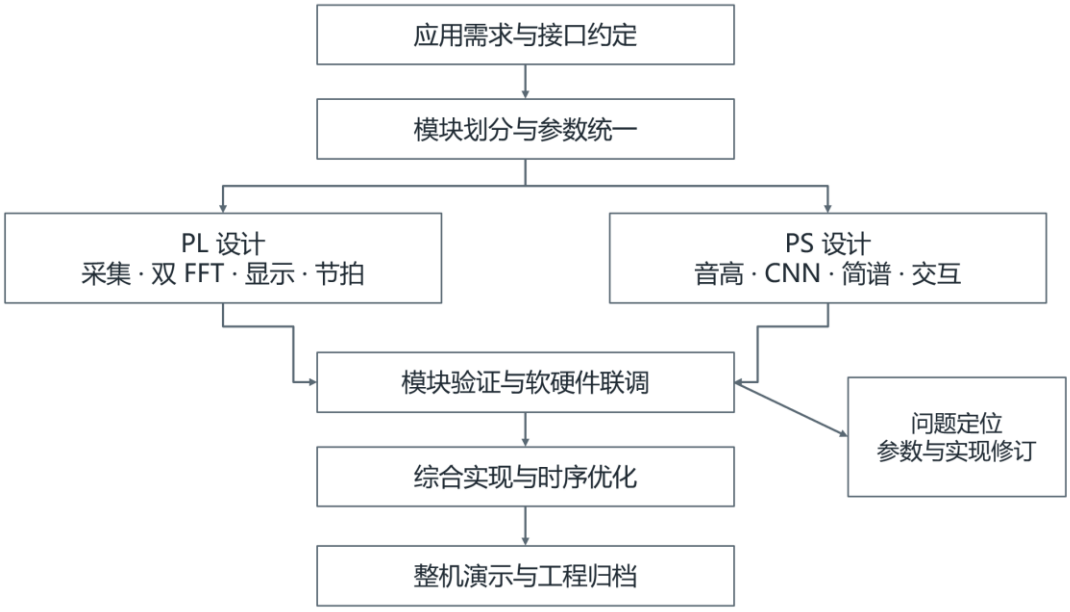


图 2　设计流程图

第二部分　系统组成及功能说明

2.1 整体介绍

系统由音频输入、片上计算、显示交互与节拍输出四部分组成[2]。ES8388 提供模拟音频接口，PL 完成流式前端，PS 执行音高与音色分析；共享 DDR 连接双路 DMA 与显示帧缓冲，形成单板闭环。

主 AXI 控制与信号处理逻辑运行于 50 MHz，音频主时钟约为 12.288 MHz，液晶像素时钟约为 33 MHz。PS 通过 AXI4-Lite 配置外设，两路音频 S2MM 和显示 VDMA 经高性能存储通路访问 DDR。

音频输入与节拍输出共享 ES8388 的转换能力；I2S RX 提供串行时钟，PL 节拍核心生成 DAC 数据。PS 仅下发节拍参数并绘制界面，时间规律稳定的输出任务由硬件持续执行。

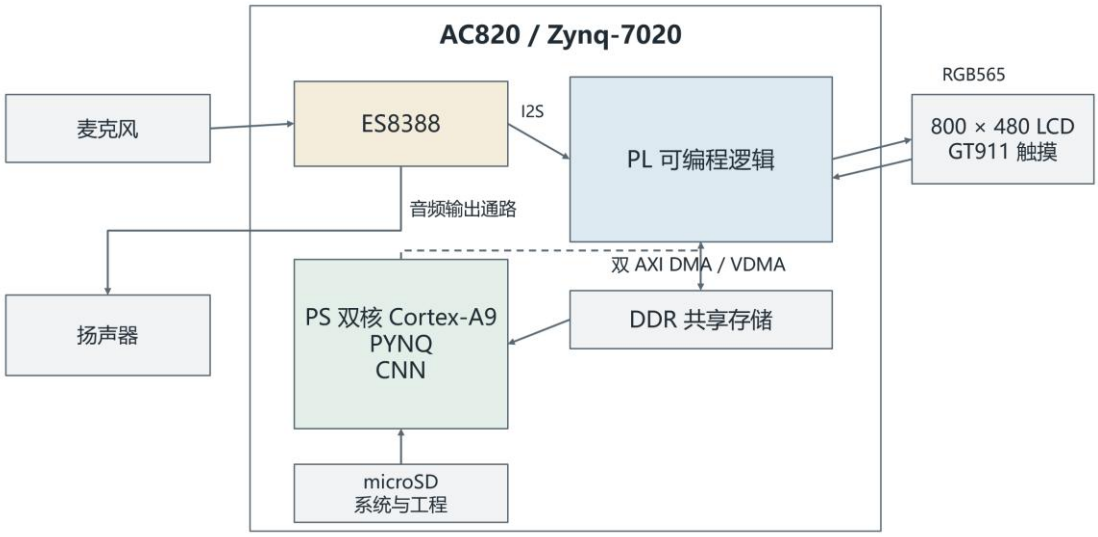


图 3　硬件整体架构

2.2 硬件系统介绍

2.2.1 硬件整体介绍

AC820 采用 XC7Z020CLG484-2，集成双核 Cortex-A9、53,200 个 LUT、106,400 个触发器、140 个 36 Kb BRAM 和 220 个 DSP 单元。板载音频、DDR 与显示接口分别承担声音转换、共享缓存和交互输出。

2.2.2 机械设计介绍

整机采用开发板与 800×480 触摸屏组合布局，音频输入、LINE OUT 和电源接口

集中于板侧，LCD 通过排线连接。结构便于展示信号流向、操作交互及独立部署，外接扬声器完成节拍听觉反馈。

2.2.3 电路各模块介绍

1. **音频前端模块。**ES8388 完成音频 A/D 与 D/A 转换，七位 I2C 地址为 0x10。PL 导出的开漏控制接口用于寄存器配置与引脚读回；输入电平、采样格式和模拟通道在启动阶段统一设置，使分析与节拍输出共享完整的音频前端[6]。

2. **I2S 接口。**I2S 接收器输出带通道标识的 AXI-Stream，单声道选择器取 TID0 的 16 位 PCM。接收器统一提供 BCLK/LRCLK，节拍核心在同一串行时钟下发送 DAC 数据，避免重复时钟驱动。

3. **显示屏接口。**显示屏采用 800×480、RGB565 格式，PS 绘制 800×480 的 BGR 三字节图像，并更新 DDR 显示帧缓冲；PL 侧 VDMA 从 DDR 读取图像，经视频时序与颜色位宽适配后，以 RGB565 输出至 LCD。界面绘制负责页面内容，硬件视频通路负责持续扫描，使音频分析与屏幕输出按各自任务组织。

4. **触摸接口。**GT911 提供触摸坐标与触点状态，PS 通过软件 I2C 访问并完成页面和控件响应。触摸控制独立于音频特征通道，支持页面切换、节拍调节和记录操作[7]。

5. **AXI 互连。**AXI 通用主口连接控制寄存器，高性能端口承载双路音频 DMA 与视频存储访问。数据搬运交由 DMA 执行，处理器围绕完整特征帧组织算法与显示任务[3]。

表格 2　作品主要接口

接口	已实现用途
I2S	ES8388 采样接收及节拍 DAC 数据发送
I2C / GPIO	ES8388 配置、GT911 触摸访问
AXI4-Lite	PS 配置 DMA、I2S、视频及节拍相关控制寄存器
AXI DMA S2MM	将两路特征包写入 DDR
AXI VDMA MM2S	从 DDR 读出显示帧缓冲

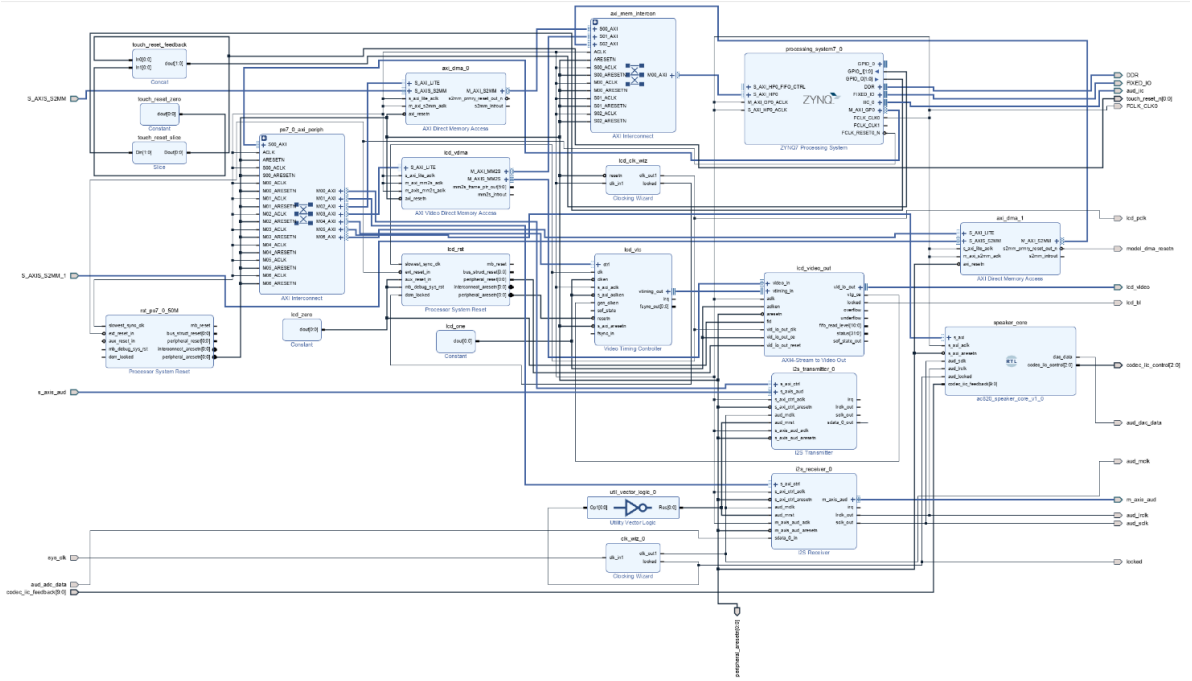


图 4　Block Design 框图

2.3 软件系统介绍

2.3.1 软件整体介绍

软件由 PL RTL/IP、PS Python 和本地 C 算法库构成，分别对应确定性数据前端、业务编排与高频数值运算。模块边界与硬件数据流一致，便于独立验证及功能扩展。

PYNQ 完成 bit/hwh 加载、寄存器访问及物理连续缓冲管理。run_live_system.py 统一管理 overlay 和硬件生命周期，页面读取共享结果，保证双 DMA 的唯一接收所有权[5]。

audio_top.v 集成单声道选择、FIR、重叠缓存、双路 FFT、打包、视频与节拍模块。源码与 BD、IP 配置、窗系数及约束一同归档，确保接口和实现依赖可追溯。

接收、音符分析、模型推理及绘图通过线程和有界队列解耦。待处理模型任务保留较新完整段，结果缓存以样本时间关联不同更新周期，避免后台任务积压影响交互。

核心分析与显示均在板端本地执行，网络承担部署与维护用途。C 库处理音高与音符，NumPy 执行卷积推理，Python 负责调度和绘制，兼顾运行效率与算法可调整性。

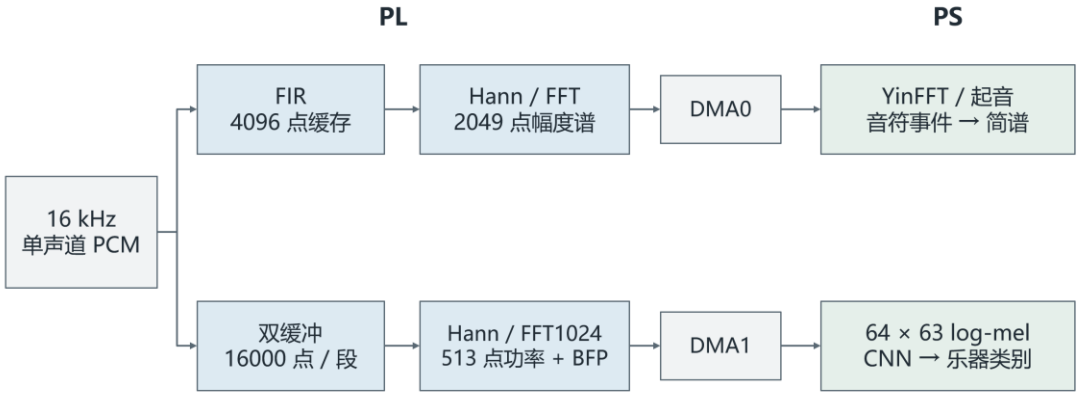


图 5　双路频谱与分析任务分工

2.3.2 软件各模块介绍

1. **单声道选择模块：**选择 TID0 并取数据位 [27:12]，形成 16 位单声道 PCM、有效脉冲与采样计数，为双路特征建立统一时间基准。
2. **FIR 与缩放模块：**音高支路执行格式适配、FIR 低通与定点缩放；模型支路直接取选定通道的 PCM，保持特征来源一致。
3. **连续缓存模块：**音高支路以历史存储和窗口快照支持连续更新；模型支路以两个 16000 点存储体交替采集与处理，溢出按完整单元计数。
4. **分帧模块：**音高长窗以 512 点帧移更新；模型通路按一秒段、256 点帧移生成 63 帧，使用段内居中零填充，对应固定模型输入。
5. **Hann 窗模块：**两路采用周期 Hann 窗，系数为 $w[n]=0.5-0.5\cos(2\pi n/N)$，$n=0\ldots N-1$。加窗减小截断带来的旁瓣泄漏，使频谱结构更集中；窗索引与有效数据握手同步，保证每个样本乘以对应位置的系数。

   我们保留音高长窗以观察低频周期，通过重叠提高更新密度；模型支路保持训练使用的短窗与补零方式。这样分别处理两种时间尺度，而不是用同一个窗长兼顾所有任务。
6. **频谱计算模块：**4096 点 FFT 保留 2049 点幅度谱；1024 点 FFT 计算实部与虚部平方和，保留 513 点功率及块浮点指数。PS 恢复功率尺度后提取梅尔特征[4]。
7. **协议与打包模块：**DMA0 每帧 8260 B，包含 16 个头字与 2049 个幅度字；DMA1 每帧 2088 B，包含 9 个头字与 513 个功率字。帧头描述位置与状态，TLAST 标识末拍。

   在缓存容量与结果新鲜度之间，我们选择缓存保护正在发送的完整帧，满载按整帧

丢弃并计数。完整谱独立保留，基频分析使用全谱信息。

8. **DMA 接收模块：**各接收器独占 S2MM 与缓冲，核对长度、头部及连续性，并同步缓存。DMA 就绪后启用采集。退出依次停止采集、DMA 和处理线程，再释放缓冲；异常保留状态，避免仍在访问的内存被释放。
9. **模型段组装模块：**按段号和帧序号 0～62 组装 513×63 功率矩阵，残缺段作废；有界队列与样本时间约束结果新鲜度。
10. **预处理与推理模块：**

1) **特征计算。** PS 利用每帧 BFP 指数 e 恢复功率尺度：P 恢复=P 输出×2^(2e)/32768²，再以固定 64×513 梅尔矩阵将每帧功率映射为 64 个频带。63 帧组成完整 64×63 时频矩阵，计算 $10\log_{10}(\max(10^{-10}, \text{mel}))$后减去全段最大值，以 clip((相对 dB＋80)/80，0，1)映射到[0,1]。一秒输入、1024 点周期 Hann、256 点帧移及居中零填充共同定义模型特征，矩阵和归一化方式保持配套。

2) **卷积网络。**64×63×1 输入依次经过三组 3×3 卷积、ReLU 和 2×2 最大池化，通道数分别为 16、32 和 64；随后使用全局平均池化得到 64 维音色表示，再经三分类全连接和 Softmax 输出原始概率。卷积学习频带分布及其时间变化，池化压缩特征尺寸，全局平均池化将时频特征汇总为分类表示。

3) 轻量适配。适配保留 23296 个卷积参数，只更新 64×3 权重和 3 个偏置，共 195 个分类头参数，总参数量仍为 23,491。模型在 PS 以 NumPy 执行，分类及支持判定共用 64 维表示，不重复运行卷积前端。原始 CNN 概率和最终判定来源分别保存。

表格 3　网络结构表

层级	操作	输出形状
输入	log-mel	64×63×1
第一组	3×3 卷积/ReLU，2×2 池化	32×31×16
第二组	3×3 卷积/ReLU，2×2 池化	16×15×32
第三组	3×3 卷积/ReLU，2×2 池化	8×7×64
汇总	全局平均池化	64
输出	全连接/Softmax	3 类

11. **音高、起音与音符跟踪模块：**

1) **基频估计。**乐器声音由基频及多个谐波组成，最大幅值频点可能落在泛音上，因此本系统不直接将最强峰作为最终音高。YinFFT 将幅度谱转为加权功率表示，构造归一化的周期差分曲线，在设定的周期范围内寻找符合条件的谷值，并对局部谷值作插值以得到周期 τ，按 $f_0=fs/\tau$ 换算基频。当前实现先对各候选谷值插值，再进行选择，从而改善整数周期位置对候选排序的影响。

2) **起音检测。**系统比较相邻帧对数幅度谱，只累计上升部分，形成谱通量 F[t]=平均值{max(0，log(1＋A[t,k]/Aref)−log(1＋A[t−1,k]/Aref))}。结合能量门限与近期通量变化识别新发声，从而为同音重奏提供区别于持续音的依据。音高回答“当前是什么音”，起音回答“是否开始了新一次发声”。

3) **音符跟踪与简谱。**音符状态机包含空闲、候选和活动三种状态，通过连续帧确认开始与换音；静音、音高变化或同音重奏触发相应 NOTE_ON/NOTE_OFF 事件。音符以 MIDI、起止样本和持续时长保存，并按 $m=\mathrm{round}(69+12\log_2(f_0/440))$映射音高，再依据 C 大调规则生成简谱数字、升号及八度标记。这样将重复帧整理为具有起止关系的单旋律记录。

团队先比较 TWM、谱显著性和 YinFFT，再根据强泛音与不同音高的固定输入结果采用 YinFFT 为默认路径。PL 仍承担连续频谱计算，PS 保留可调整的周期和音符决策，使算法比较能够复用硬件前端[10]。

第三部分　完成情况及性能参数

3.1 整体介绍

整机已完成声音输入、硬件双谱、音高与音色分析、触摸显示及节拍输出的系统集成。实物与界面共同展示了单块 Zynq 平台从信号感知到练习反馈的完整链路。

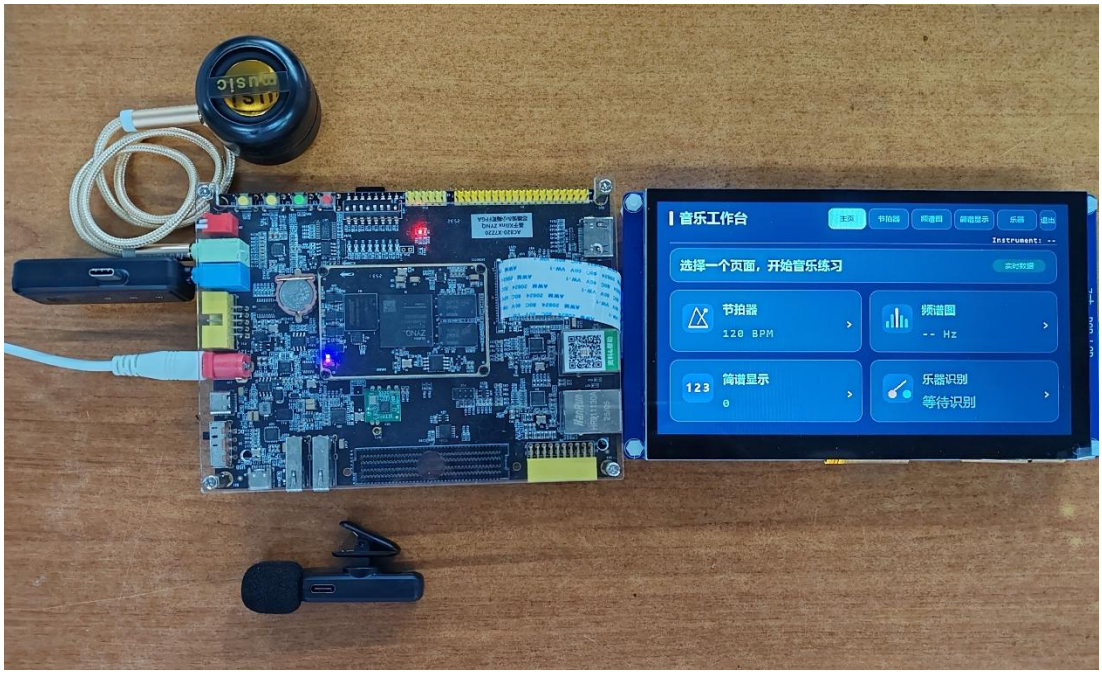


图 6　系统实物正面图



图 7　系统实物斜 45°图

3.2 工程成果

3.2.1 机械成果

开发板、触摸屏与外接音频设备构成可独立演示的终端。板级布局与接口连接如图 8 所示，兼顾现场操作和模块维护。

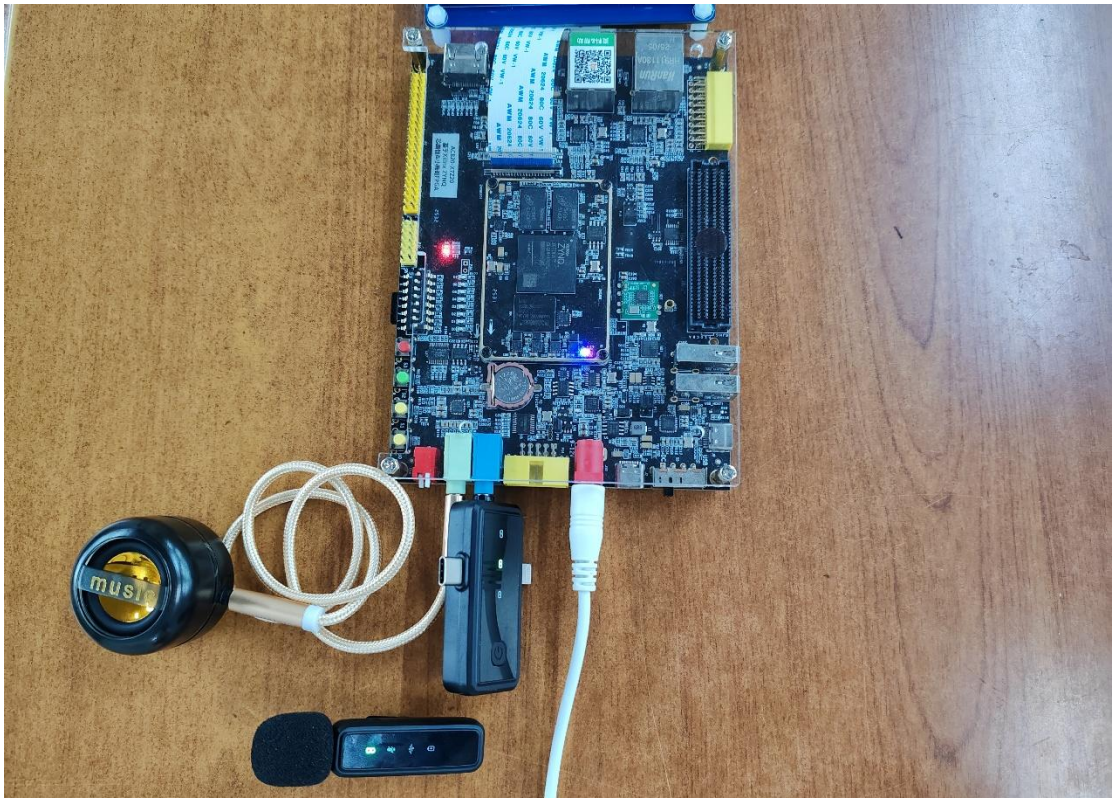


图 8 板级实物图

3.2.2 硬件成果

电路成果包括统一顶层、Block Design、管脚约束以及双路 FFT/DMA、视频和节拍输出链路。合并实现仅占用 26.12% LUT 和 12.27% DSP，兼顾功能集成与资源余量；时序方面，**WNS = +7.336 ns、WHS = +0.040 ns，TNS 与 THS 均为 0 ns**，在所设约束下无违例路径。保持裕量仅 0.040 ns，已接近零；后续任何 PL 改动都需重新签核时序，不能沿用本次结果。

3.2.3 软件成果

软件界面围绕练习任务组织为五个页面，主页统一进入各功能。律动频谱、当前音、滚动简谱与乐器识别各有独立视图，触摸操作与后台分析并行运行。



图 9　终端主页

主页提供各功能入口及运行信息，便于练习过程中快速切换任务。



图 10　终端节拍器页面

节拍器由 PS 设置 BPM、拍号和启停，PL 依据主时钟生成拍点及节拍音频。设主时钟为 fclk，速度为 BPM，计数周期按 round(60fclk/BPM)配置；硬件以组内拍序安排重音，经 I2S 数据发送、ES8388 DAC 和 LINE OUT 输出。参数交互与持续计时分开，避免用界面刷新承担节拍定时。



图 11　终端律动频谱页面

律动频谱复用音高支路的完整幅度谱，将频点分组后映射为相对 dB 及柱高，形成随输入变化的柱状效果。界面按当前结果缓存绘制，不额外建立音频波形通路。

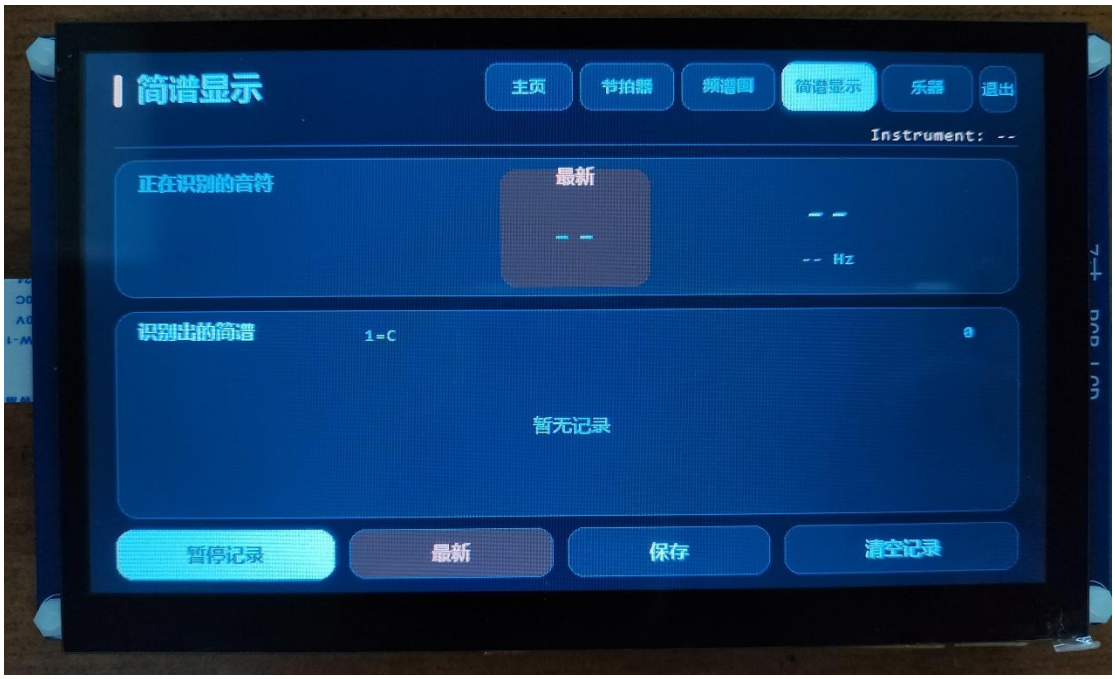


图 12　终端简谱识别页面

简谱页面显示当前音与音符历史，并支持记录操作，使演奏过程能够被连续回看。



图 13　终端乐器识别页面

乐器页面以图标与类别呈现三类音色识别结果，减少信息阅读负担。

3.3 特性成果

特性成果从音高验证、优化对比、资源时序与模型部署成本四个方面归纳，分别见表 4 至表 7。32 ms 为更新间隔，256 ms 为观察窗，二者均不等同于整机延迟；本轮未做端到端延迟测量。

下文 V1 指 2026-09 的 NSynth 基线（int8 TFLite，30.5 KB），R6 指 2026-11 的现场适配版（float32 NumPy，93.7 KB）；两者网络结构与 23,491 个参数完全相同，差异只在分类头与封装。

表格 4　音高与音符验证

测试项	输入	期望	实测	结论
纯音落点	500 Hz	bin 128	bin 128	通过
纯音落点	1000 Hz	bin 256	bin 256	通过
六音阶	C4 – A4，500 ms/音	6 个音，顺序 1 – 6	6/6	通过
同音重奏	C4×3，间隔 200 ms	3 个独立 note_id	3/3	通过
静音	无输入 10 s	0 个事件	0	通过

测试项	输入	期望	实测	结论
真实钢琴	C2–C6 分组（5 组×12 音，共 60 音）	顺序正确	57/60	有 3 处漏/错

表格 5　优化前后与功能对比

优化对象	基线	优化后	采用的手段
音高分析更新	4096 点无重叠，256 ms	32ms 更新	连续缓存 + hop512
音高判断	最强谱峰	YinFFT + 音符事件链	完整谱 + 三状态机
手机现场录音分类	21/55（38.2%）	42/55（76.4%）	冻结卷积 + 分类头
实体钢琴回归	22/49（44.9%）	37/49（75.5%）	分类头 + 钢琴支持
新录音回放	13/25	24/25（96.0%）	加入钢琴支持
外部三条独奏	1/3	3/3（100%）	R6 部署与适配

各分类行沿用附录已有测试组名称及样本数。三条独奏按逐文件结果展示，与 55 段现场回放分表或分行呈现。网络推理、采集窗口和界面显示各按自身任务描述，不把 32ms 换写为整机总延迟。

表格 6　合并设计的器件资源与时序

项目	使用量	器件总量	占用率
LUT	13897	53200	26.12%
FF	20874	106400	19.62%
BRAM36 等效	45	140	32.14%
DSP	27	220	12.27%
WNS（建立）	—	—	+7.336 ns
WHS（保持）	—	—	+0.040 ns
TNS / THS	—	—	0 ns / 0 ns

合并设计在上述资源内集成双路频谱、显示和节拍输出。完整幅度谱服务音高、起音与律动，卷积特征服务分类及钢琴支持判定，重复业务通过共享中间结果实现。分类适配在 PS 侧更新小规模参数，不为适配单独增加 PL 卷积计算链，体现了性能目标与片上资源预算的协调。

表格 7　模型适配的计算与部署成本

指标	V1 基线	R6
网络总参数	23,491	23,491
卷积参数	23296	23296，冻结
本次适配参数	—	195
网络计算量	9.6M MACs	9.6M MACs
部署总占用	159.2KB	264.3KB
推理部署	int8 TFLite	float32 NumPy

其中 195 是参与适配的分类头参数数目，不是模型总参数；部署文件体积属于软件存储成本，与 BRAM 和 DSP 资源表分别解释。

我们首先通过缓存和帧移改变更新节拍，而不缩短低音分析使用的观察窗；随后在 PS 比较基频算法，保持硬件频谱前端可复用。音色适配也保留卷积特征，只训练 195 个头部参数，并利用同一表示补充钢琴支持判定。由此，优化围绕已有计算结果和软件决策展开，避免为每项功能建立新的硬件处理链。R6 改用 NumPy 完成板端部署，文件占用由 159.2KB 增加至 264.3KB，作为兼容部署与现场适配的实际成本一并列出。

表格 8　作品最终成果呈现

	
1. 节拍器：30～240 BPM，支持 2/4、3/4、4/4、6/8 拍及启停控制。	2. 律动频谱：以真实幅度谱生成柱状显示，直观反映演奏的频谱变化。
	
3. 单旋律简谱：识别当前音高并按 1=C 映射为简谱数字，记录音符起止，生成滚动简谱。	4. 音色识别：钢琴演奏输入对应的类别显示。
	
5. 音色识别：小提琴演奏输入对应的类别显示。	6. 音色识别：吉他演奏输入对应的类别显示。

第四部分　总结

4.1　可扩展之处

现有模块化结构为扩展乐器类别、调音辅助与练习评分提供了基础。保留既有采样与特征接口，可通过扩充独立录音资料和分类策略覆盖更多音区与演奏方式；多声部识别与和弦分析可进一步利用片上资源余量展开。

终端还可拓展通过网络导出练习记录、同步参考曲谱与分析结果，面向课堂和家庭形成更完整的练习服务。结构设计可围绕屏幕、麦克风和扬声器进行一体化封装，增强实际使用中的布置便利性。

4.2　心得体会

项目最初源于音乐学习中的一个需求：能否在演奏时同时看到声音变化、判断乐器音色，并将旋律记录为简谱？围绕这一场景，我们把系统划分为声音采集、频谱计算、音高与音色分析、显示交互几个环节，先建立单音检测基线，再逐步接入模型和触摸屏。

起初，我们认为找到频谱中幅值最大的点就能得到音高。调研乐理和声学知识后，我们认识到基频、泛音与听感音高的关系；实际乐器测试也暴露出泛音强于基波、低音被识别为高八度的问题。我们先加入 Hann 窗改善频谱泄漏，但错音仍然存在，说明泄漏与基频判定是不同的问题。因此，我们将完整频谱传给 PS，对比 TWM、谱显著性和 YinFFT。在强第三谐波 A2 和 E5 等测试中，依据正确性、有效输出及计算开销，选择 YinFFT 作为默认音高算法，而不是继续依赖最强峰。

随后，快速弹奏中的漏音让我们重新审视时间处理。4096 点窗口有利于观察低音，却需要积累较长时间。我们保留窗口长度，引入连续缓存和重叠分帧，将目标更新间隔缩短到 32ms。但同音重奏和延音重叠仍不能仅靠逐帧音高解决，于是进一步加入起音检测和音符状态机，通过音阶、重复音、渐强等测试检查切分行为。这使我们理解：更新更快、音高更准与音符切分正确，是相互关联但需要分别验证的目标。

音色识别遇到的主要困难是训练特征与板端数据的一致性，以及现场声音的变化。我们逐项核对窗函数、补零、帧数、功率缩放和归一化，并比较相同特征在 PC 与 ARM 上的结果，再开展分类头适配。由此学会先区分数据错误与模型泛化问题，也意识到训练素材回放不能替代独立测试。

软硬件分工同样经历了取舍。我们将连续采样、缓存、加窗和 FFT 放在 PL，利用硬件流水处理；将需要反复调整的基频决策、音符跟踪和分类放在 PS。双路频谱分别满足音高分析与模型输入需求，后续优化则从前端电平、数据连续性、算法决策和显示调度共同入手。

这次实践让我们从单个模块能否运行，转向判断整条链路是否满足任务目标。面对错音，我们分别检查泄漏、泛音与音符切分；面对响应慢，我们拆分观察窗、更新网格和界面任务；面对分类偏差，我们先对齐数值，再调整决策。大模型为这些判断提供研究线索、代码候选和验证支持，我们负责提出路径、比较取舍并依据实际结果决定采用方案。我们将有效的检查步骤进一步整理为可复用技能，使经验能够通过脚本和输入契约传递给其他使用者。

第五部分 参考文献

[1] 赛事组织方. 2026 年 AMD 选题指南[Z]. 团队提供版本, 3.3.3—3.3.5 及第六节.
[2] AMD. Zynq-7000 SoC Technical Reference Manual: UG585[Z]. 工程所用器件技术参考.
[3] Xilinx. AXI DMA LogiCORE IP Product Guide: PG021[Z]. Vivado 2020.2 配套文档.
[4] Xilinx. Fast Fourier Transform LogiCORE IP Product Guide: PG109[Z]. Vivado 2020.2 配套文档.
[5] PYNQ Project. PYNQ Documentation, Release 2.7[EB/OL]. https://pynq.readthedocs.io/en/v2.7.0/.
[6] Everest Semiconductor. ES8388 Low Power Stereo Audio CODEC Datasheet[Z].
[7] Goodix. GT911 Touch Controller Programming Guide[Z].
[8] DE CHEVEIGNÉ A, KAWAHARA H. YIN, a fundamental frequency estimator for speech and music[J]. Journal of the Acoustical Society of America, 2002, 111(4): 1917–1930. DOI: 10.1121/1.1458024.
[9] ERONEN A, KLAPURI A. Musical instrument recognition using cepstral coefficients and temporal features[C]//Proceedings of the 2000 IEEE International Conference on Acoustics, Speech, and Signal Processing. 2000, 2: II-753–II-756.
[10] BOGDANOV D, WACK N, GÓMEZ E, et al. Essentia: An audio analysis library for music information retrieval[C]//Proceedings of the 14th International Society for Music Information Retrieval Conference. 2013: 493–498.

附录　优化过程

1　结论摘要

两个版本的网络结构与参数量完全相同（23,491 参数、9.6M MACs），差异只在分类头与封装方式：

- V1：NSynth 数据集训练，int8 量化，tflite 部署。
- R6：冻结三层卷积，仅重训 195 个头部参数（55 段手机录音），改用 float32 numpy 推理，并增加钢琴模板匹配。

效果呈明确的场景权衡，不是全面升级：

维度	V1	R6	判断
外部真实独奏（3 段）	1/3	3/3	R6 修复
自建 13 音高单音集（36 条）	35/36	28/36	R6 退化
部署体积	159 KB	264 KB	V1 更小

2 版本演进

阶段	时间	数据	训练范围	部署格式
V1 基线	2026-09-23	NSynth acoustic 三类（训练 4,224 条，每类 1,408 均衡；测试 264 条，每类 88）	全网络训练（40 epoch）	int8 tflite，30.5 KB
R6 候选	2026-11-01	55 段手机录音 + 70 条旧参考回放	仅 Dense 层 195 参数	float32 npz，93.7 KB

R6 的触发原因：V1 在真实录音上出现系统性误判（钢琴、小提琴均被判为 guitar），需针对现场采集分布做适配。

3 资源占用对比

指标	V1 基线	R6 候选	变化
总参数量	23,491	23,491	不变
卷积层参数	23,296	23,296（冻结，逐位相同）	不变
分类头参数	195	195（已重训）	数值改变
计算量（MACs）	9.6M	9.6M	不变
模型权重体积	30.5 KB	93.7 KB	×3.1
特征矩阵 mel_matrix	128.4 KB	128.4 KB	不变（SHA256 相同）
参考向量库	—	41.4 KB（70 条）	新增
元数据	0.3 KB	0.8 KB	+0.5 KB
部署总占用	159.2KB	264.3 KB	×1.66
数值精度	int8	float32	—
推理框架	tflite_runtime	纯 numpy	—
运行时依赖	tflite_runtime	仅 numpy	依赖减少

说明：R6 改用 numpy 浮点推理的原因——Zynq-7020 的 Cortex-A9 仅支持 VFPv3，Coral 的 tflite_runtime wheel 按 VFPv4 编译，板上 invoke() 报 Illegal instruction。numpy 重写可绕开二进制兼容问题，代价是体积增大与浮点运算开销。

4 性能对比

4.1 同源单音

测试集	样本数	V1	R6	说明
NSynth 同源测试集（三类，各 88 条）	264	94.32%	—	V1 的 int8 量化后指标（fp32 为 95.45%）
自建 13 音高单音集（G3~G5 为主）	36	97.2%（35/36）	77.8%（28/36）	R6 退化 7 条

R6 退化 7 条：吉他 A4/C5/E5/F4/G4/G5 六条被判 violin，violin_C4 被判 guitar。另有 guitar_E2 在 V1/R6 均错，R6 置信度由 62.9% 升至 93.8%，**不计入退化**。

4.2 外部真实独奏

文件	真实标签	V1	R6
piano_solo.wav（Satie，30 s）	piano	guitar 80.8% ✗	piano 93.3% ✓
guitar_solo.wav（Gravity Sound，16.7 s）	guitar	guitar 95.1% ✓	guitar 86.9% ✓
violin_solo.wav（Bach，25 s）	violin	guitar 92.8% ✗	violin 97.5% ✓
合计	3	1/3（33.3%）	3/3（100%）

4.3 现场录音与实体乐器

测试集	样本数	优化前	优化后
手机现场录音分组留出	55	21/55（38.2%）	42/55（76.4%）
实体钢琴回归（合计）	49	22/49（44.9%）	37/49（75.5%）
低音区	31	17/31	27/31
中音区	13	5/13	7/13
高音区	5	0/5	3/5
新录音回放（含钢琴模板）	25	13/25	24/25

其中录音为手机外放，固定音量和距离，实体乐器为近距离弹奏，均按音高分组留出验证。中音区受手机录音信噪比与训练样本分布影响，高音区受 16 kHz 采样下谱密度下降限制；上述数字不作为全音域准确率，后续需补充独立录音与增益校准。

5 优化手段

5.1 冻结卷积、重训分类头

- 三层卷积权重完全冻结，实测与 V1 最大绝对差 0.00000000（逐位相同）。
- 仅优化 Dense 层 64×3 权重与 3 个偏置，共 195 参数。
- 训练 800 步 Adam，学习率 0.02，向原头部 L2 约束 0.001，标签平滑 0.02。
- 用 70 条旧参考 embedding 作 25% 权重的回放，防止特征空间漂移。

5.2 钢琴模板匹配（后处理）

钢琴支持模块复用 CNN 的 64 维音色表示，经过中心化和归一化后，与钢琴正模板及吉他、小提琴负模板比较。满足相似度门限及相对间隔条件时，可将原判为吉他或小提琴的结果修正为钢琴，补充现场钢琴的识别。程序保留原始 CNN 概率，并单独记录支持判定来源，不重复执行卷积。

- 从 1 段新录音取 23 个正模板，配 93 个吉他/小提琴负模板。
- 采用中心化余弦相似度（centered_cosine_local_piano_support_v1），判定阈值：相似度下限 0.85、负样本排除 0.02、相对间隔 0.05。
- 作用：对 CNN 判为吉他/小提琴的样本做二次判定，将满足钢琴模板相似度与相对间隔条件的样本修正为钢琴，降低真实钢琴的漏检；原判为钢琴的结果不再重复判定。

5.3 封装替换

- int8 tflite → float32 numpy，解决 A9 的 VFPv4 指令不兼容问题。
- 卷积与池化用 numpy 逐层复刻，接口与 V1 保持一致。

6 权衡分析

R6 在保留卷积特征的情况下调整分类边界，现场 55 段回放由 21 段判对提高到 42 段，实体钢琴回归由 22/49 提高到 37/49；在 36 条单音集上，结果由 35/36 变为 28/36。这说明本次适配改变了不同输入分布上的判别表现。我们因此按输入类型分别列出结果，并把现场收益、同源回归和部署成本共同作为评价依据。最终实现采用冻结卷积、分类头适配和钢琴支持，保持现有网络计算量与硬件结构。

附录　大模型协作记录与技能包的提炼过程

本版以团队的架构思考、方案取舍和软硬件协同实践为主线，详细摘录与证据见ai_collaboration/。

1 团队主导的方案形成

本项目中，我们负责场景分析、架构与实现路径的设想、备选方案比较和最终决策，并组织模块分工、整合与上板测试。大模型作为辅助工具，围绕我们提出的问题提供文献与开源线索、原理解释、候选代码和验证脚本，帮助核对接口、分析测试结果和修订细节。候选方案是否采用，由我们结合目标、已有工程和实测结果决定。

我们首先从模型接口反推数据通路：既然音色模型需要功率谱，能否让 PL 先完成频谱计算，再通过 DMA 把结果送到 PS？随后进一步比较峰值搜索应放在 PL 还是 PS、两路分析是否共用 FFT，以及裸机程序如何迁移至 PYNQ。讨论逐步形成了当前分工：PL 承担连续采样、缓存、加窗和双路 FFT；PS 利用完整频谱完成基频、起音和音符决策，并组织模型推理与本地界面。Python 负责调度，本地 C 库保留音高和音符核心。

这一划分是在需求、候选分析与验证之间逐步收敛的工程选择。我们既考虑硬件加速，也考虑模型特征一致性、分析更新节拍和软件调整成本，没有把“更多运算放在 PL”直接等同于更优，也没有把当前双路实现宣称为资源最优。

2　三个体现团队决策的协作案例

2.1 C01 音高算法比较

我们提出的猜想或判断：提出比较最强峰、YIN 或 HPS 等路线，要求研究依据和固定输入验证。

大模型提供的辅助：提供候选原理与实现支持，形成 TWM、SAL、YinFFT 的对比代码，辅助解释泛音误判和输出差异。

我们的取舍与验证：根据 A2 强第三谐波及 E5 等实测，工程采用 YinFFT 为默认路径，保留其他候选和 PL 参考峰；不把个别样例外推为全音域准确率

在架构取舍中，我们追问“双 FFT 的建议是否已有资源评估支撑”，使临时验证策略与资源最优结论分开；在降低等待时，我们区分 4096 点观察窗与 512 点帧移，避免将 32ms 更新间隔当成 32ms 总延迟；在算法选择中，我们将基频、起音和音符切分分别验证，避

免仅凭谱峰稳定就认为简谱已正确。这些过程体现了对数据通路、任务节拍和软硬件边界的持续判断。

2.2 C02 连续缓存与重叠分帧

我们提出的猜想或判断：从实际演奏反应慢出发，提出优先优化连续缓存和重叠分帧，而不只调整搜索算法。

大模型提供的辅助：辅助展开历史缓存、窗口调度、Hann 握手和整帧背压规则，提供参考计算与验证方法。

我们的取舍与验证：采用 4096 点窗、hop512，交队友实现并组织验收；历史单支路测得 31.244 帧/秒，同时保留 256ms 窗长和短音过渡问题的说明。

2.3 C05 软硬件划分与双路架构

我们提出的猜想或判断：提出 PL 计算模型功率谱；追问峰值搜索位置、双 FFT 能否共用，以及资源依据。

大模型提供的辅助：梳理不同窗口与特征需求，展开数据格式、模块接口和候选实现；辅助核查工程与所选路径是否一致。

我们的取舍与验证：决定先做双路验证，4096 点完整谱交 PS 比较音高算法；模型支路保持既定功率谱接口，最终按 PYNQ 与 LCD 整合。

3 人机协作与纠正方式

协作流程为：我们提出路径假设和约束，大模型辅助调研与展开验证，团队选择候选并安排实现，再以真实输入、源码和日志核对，发现偏差后修订。

大模型参与了部分候选代码及脚本工作，相关实现贡献按实际记录说明；关键方案的采用、测试条件和验收由团队掌握。

版本加载确认和交付文件校验作为补充案例保留。它们说明辅助检查如何发现启动顺序、旧对象或文件清单等细节问题，不作为本文展示团队核心设计能力的主要案例。我们保留这些记录，是为了让方案和交付都能被复查。

4 从工程观察到可复用技能

我们围绕项目中反复遇到的问题，总结四类检查需求，并借助大模型将其整理为脚本、模板和边界说明。

团队关注的问题	提炼成果	验证与边界
PYNQ 整合后如何确认制品和硬件所有者	PYNQ 部署与 DMA 生命周期管理	文件/HWH 及模拟停止故障检查；不等同于所有板卡均已验证
完整谱是否同帧、是否连续、慢接收是否破坏数据	完整帧与背压校验	联合核查点数、bin、采样时间和参考载荷，配合历史板测及故障注入；包检查不能替代 RTL 握手验证
硬件结果和模型预处理是否保持相同数值含义	DSP 数值与模型特征对齐	冻结格式、FFT 指数、mel 与归一化约定，逐层比较同源数据；数值一致与识别准确率分别评价
优化结果和交付内容能否追溯	构建与性能证据核查	将制品与报告对应，分开局部耗时、更新间隔和整机延迟；不能替代完整重建及最终上板

我们将项目中的检查经验组织为四个技能目录，每项包含适用问题、输入契约、操作说明、脚本及示例。使用者先填写自身工程的 IP 映射、包格式或数值参数，再运行正常示例和故障示例，最后替换为实际数据。PYNQ 技能核对制品与 DMA 生命周期，帧技能检查时间和载荷连续性，数值技能比较同源特征，构建技能核对报告与制品。现有技能工具已完成 28 项主机故障注入与 9 组命令行回放，详见 skill/validation_report.md。

5 原文与证据

主案例使用本会话中团队主动提出路径、质询方案和决定执行的原文、提示词及模型回答，详见 report/ai_collaboration/conversation_excerpts.md。早期助手完整回答未全部保留，事后归纳与原文分开，没有把讨论中逐步形成的方案写成任何一方独立完成的原创，也没有把候选建议写成已经通过的结果。阶段测试范围和实际实现贡献均按现有证据说明。

附录　复现说明

1 工具链版本声明

本项目使用 Vivado 2020.2 与 Vitis 2020.2，配套 AC820 厂商 PYNQ 2.7.0 镜像。板端环境为 Python 3.8.2、NumPy 1.20.3、Pillow 7.0.0 及 Linux 5.4.0-xilinx-v2020.2。实际环境记录位于 board/environment/environment.json。

GitHub 总工程位于 final_20261104/music_analysis_terminal，包含 src、sim、build、board、data、skill 和 report。平台复现目录位于 platform_reproduction_20261104/music_analysis_terminal，包含 src、build 和 board；两份工程均从根 README.md 开始复现。上述路径为本机交付目录中的相对位置。

本文后续以解压后的 music_analysis_terminal 为工程根目录，所有 src/、build/、board/ 等路径均相对于该目录。功能验证涉及的 data/和 sim/材料从 GitHub 总工程取得。两份工程使用各自的 release_manifest.json。

2 复现前置条件

项目	要求
开发板与器件	AC820，XC7Z020CLG484-2，配套 ES8388 与 LCD/GT911
PL 工具与文件	Vivado 2020.2；同次构建的 bit/hwh、源码、IP、BD、XDC 与报告
板端环境	PYNQ 2.7.0；Python 3.8.2；NumPy 1.20.3；Linux 5.4.0-xilinx-v2020.2
电脑环境	Windows、Python 3.12；完整主机验证另需 NumPy；Vivado/Vitis 2020.2 及 Zynq-7000 器件支持
系统镜像	独立镜像附件 ac820_pynq_2_7_0_image.zip，解压得到 ACZ820_ZYNQ-2.7.0.img
工程包	music_analysis_terminal 目录；根 README.md 为统一入口，release_manifest.json 用于文件校验

3 源码构建

3.1 打开命令窗口

先在电脑解压工程包，将 music_analysis_terminal 放到 D:/music。以下以该位置为例，在 PowerShell 执行。若实际目录不同，请替换第一行路径。

Set-Location -LiteralPath 'D:\music\music_analysis_terminal'

python --version

3.2 创建 Vivado 工程

在同一 PowerShell 窗口执行。第一行替换为本机 Vivado 2020.2 的实际安装目录。

$env:MUSIC_VIVADO_ROOT = 'D:/Xilinx/Vivado/2020.2'

python -B .\build\scripts\rebuild.py --create-only --out D:\music_create_01

出现 CREATE_PROJECT_OK 后，工程位于 D:\music_create_01\project\audio_merged.xpr。

3.3 综合、实现及文件生成

继续在 PowerShell 执行：

python -B .\build\scripts\rebuild.py --out D:\music_build_01 --jobs 4

出现 REBUILD_OK 后，查看本次结果：

D:/music_build_01/artifacts/　audio_merged.bit、audio_merged.hwh、audio_merged.xsa

D:/music_build_01/reports/　综合、资源、时序与 DRC 报告

D:/music_build_01/build_receipt.json　构建记录及制品比较结果

以上输出目录使用尚不存在的新目录；再次构建时将编号改为 02。

原交付目录内 build/project/readme.md 说明工程生成方式，实际 audio_merged.xpr 由上述脚本在新输出目录创建。运行配套 bit/hwh 位于 board/runtime 及 board/artifacts。

3.4 检查文件一致性

在电脑工程根目录执行：

python -B .\build\scripts\verify_release.py

出现 RELEASE_VERIFY_OK 表示当前交付目录的发布清单及配套运行制品检查通过。新构建制品与冻结制品的字节比较结果见本轮 build_receipt.json；其资源与时序由本轮 reports 中的报告说明。

3.5 编译 PS 侧 C 算法库

在同一 PowerShell 窗口执行，将前两行改为本机 Vivado/Vitis 2020.2 中相应编译器的位置。

```
$env:MUSIC_HOST_CC = 'D:/Xilinx/Vivado/2020.2/tps/mingw/6.2.0/win64.o/nt/bin/gcc.exe'
$env:MUSIC_ARM_CC = 'D:/Xilinx/Vitis/2020.2/gnu/aarch32/nt/gcc-arm-linux-gnueabi/bin/arm-linux-gnueabihf-gcc.exe'
python -B .\build\scripts\build_native.py --out D:\music_native_01
```

输出目录包含 Windows 库 libmusic_ps_host.dll、ARM 库 libmusic_ps_arm.so、host_build.log、arm_build.log 及 build_results.json。板端正式运行加载 board/runtime/dsp/libmusic_ps.so。

4 板端部署

4.1 启动开发板

1. 独立镜像附件的文件名与 SHA256 见 board/environment/image_manifest.json。将镜像解压后，使用 Win32DiskImager 把 ACZ820_ZYNQ-2.7.0.img 写入 SD 卡，写入前核对目标盘符。
2. 关闭板卡电源，设置 SD 启动模式，插入 SD 卡并连接 LCD、触摸排线。
3. 连接电源和网线，使开发板与电脑处于同一局域网，启动开发板。
4. 电脑音频输出连接板卡 LINE IN，LINE OUT 连接有源音箱或功放。

4.2 打开 Jupyter

在电脑浏览器访问板卡实际 IP 的 9090 端口。地址格式如下。

```
http://实际 IP:9090
```

厂商原始镜像的默认密码为 xilinx。

若不知道板卡 IP，可查看路由器的设备列表，或经串口终端登录板卡后执行以下命令。

```
ip -4 addr show eth0
```

读取 eth0 下 inet 后的 IPv4 地址并替换“实际 IP”。板卡与电脑应处于可互通的同一局域网。

4.3 上传工程并设置根目录

先在电脑解压平台 RAR 或总工程 ZIP，再使用 WinSCP 等 SFTP 工具将整个 music_analysis_terminal 目录上传至板卡。下列示例假定放在/home/xilinx/jupyter_notebooks 下；若位置不同，将 ROOT 改为实际路径。在 Jupyter 新建 Python 3 Notebook，执行以下代码。

```
from pathlib import Path
import os
ROOT = Path('/home/xilinx/jupyter_notebooks/music_analysis_terminal')
assert (ROOT / 'board/START_LCD.ipynb').is_file(), '请检查 ROOT 路径'
os.chdir(ROOT)
print(ROOT)
```

上述路径为板端部署位置示例。工程解压后应直接包含 README.md、src、build 和 board，避免多套一层同名目录。

4.4 检查工程与环境

在同一 Notebook 新增单元格。此时当前工作目录应为 ROOT，即 music_analysis_terminal 根目录。

```
%run build/scripts/verify_release.py
%run board/scripts/export_environment.py --out board/environment/environment_first_run.json
```

第一条命令应输出 RELEASE_VERIFY_OK，第二条命令输出环境文件保存位置。若 environment_first_run.json 已存在，改用新的文件名。

4.5 自检与启动

继续在同一 Notebook 执行，使用上一节定义的 ROOT 进入运行目录。

```
os.chdir(ROOT / 'board' / 'runtime')
%run run_live_system.py --check-only
```

出现 NATIVE_LIBRARY_SELFTEST_OK 后，在下一单元格启动：

```
%run run_live_system.py --input line --ui-stats
```

LCD 显示界面后开始播放测试音频。单元格持续显示[*]表示程序正在运行。

参数	使用方式
--input line	电脑音频线输入的默认配置
--input vitis	复现原高增益配置时替换上一项
--piano-review	可选钢琴分类复核，默认关闭；启用时在正式启动命令末尾追加

钢琴支持判定默认开启，钢琴复核默认关闭，实际测试应记录这两个配置。也可在 Jupyter 文件列表打开工程下的 board/START_LCD.ipynb，按顺序运行全部单元格。

4.6 停止与保存结果

1. 点击 Notebook 菜单 Kernel → Interrupt，等待运行结束。
2. 从 Jupyter 文件列表进入 board/runtime/music_records，下载本轮 JSON 记录。
3. 若启动时设置了--capture-label 以采集模型特征，结束后等待 CAPTURE_FINAL，并按 CAPTURE_DIR 显示的位置收集数据。
4. 保存 Notebook，连同音频文件、正确标签及屏幕实拍一起归档。

5 功能验证

本节测试音频与参考标签取自 GitHub 总工程。data 路径以总工程根目录为基准；使用平台包上板时，另从总工程下载这些测试输入并在电脑播放。音符预期标签见 data/labels/acceptance/expected.csv。

执行动作	输入或操作位置	观察结果
纯音与静音	data/audio/acceptance/pure_500_1000.wav	500Hz、1000Hz 区间的稳态谱峰接近 bin128、bin256；律动页面响应
六音阶	data/audio/acceptance/scale_c4_a4.wav	依次形成 C4、D4、E4、F4、G4、A4 音符
三次同音	data/audio/acceptance/repeat_c4.wav	生成三个独立 C4 事件
乐器回放	data/README.md 所列音频及来源	保存真实标签和系统输出类别

执行动作	输入或操作位置	观察结果
交互与节拍	触摸五页、设置 BPM 与拍号、启停	页面响应及 LINE OUT 节拍输出

表中各项为验证输入与预期结果。实际音符和分类结果由系统保存，人工记录实际声源、正确标签、电脑音量、启动参数及播放顺序，再与预期对照。

完整主机与 RTL 验证使用 GitHub 总工程。在电脑工程根目录执行以下命令，输出目录使用新的名称，Vivado 路径改为实际安装位置。

```
python -B sim/scripts/run_validation.py --out sim/results/host_replay01
python -B sim/scripts/run_rtl.py --vivado-root D:/Xilinx/Vivado/2020.2 --out
sim/results/rtl_replay01
python -B data/verify_reference_audio.py
```

已有板端实测输出位于总工程 board/results/current_r6。技能验证入口为 skill/tests/verify_skill_package.py，相关协作材料位于 report/ai_collaboration。

6 构建与运行记录

保留实际启动命令、board/environment 导出的环境文件、运行记录及对应输入。正式制品的综合实现报告位于 build/reports/frozen_20261031，20261104 独立重建报告位于 build/reports/rebuild_20261104。每次新构建的报告与制品由--out 指定目录保存，并按构建批次登记。